

Evaluación de LLMs — Benchmarks, Datasets, Métricas y Frameworks

Taller 5 · Módulo 04: Evaluación de Modelos

Dr. Vicente Machaca Arceda | UTEC

Tabla de Contenidos

1. [Introducción: ¿Por qué Evaluar es Difícil?](#)
 2. [Paradigmas de Evaluación](#)
 3. [Tipos de Evaluación: Capacidades vs. Tareas](#)
 4. [Métricas Clásicas de NLP](#)
 5. [Benchmarks de Conocimiento y Razonamiento](#)
 6. [Benchmarks de Código, QA y Seguridad](#)
 7. [Evaluación Humana y Arenas](#)
 8. [LLM-as-a-Judge](#)
 9. [Métricas para RAG](#)
 10. [Métricas para Agentes](#)
 11. [Frameworks de Evaluación](#)
 12. [LangSmith para Evaluación](#)
 13. [Leaderboards Públicos](#)
 14. [Dimensiones Más Allá de la Precisión](#)
 15. [Buenas Prácticas y Resumen](#)
 16. [Ejercicios del Taller](#)
 17. [Glosario Rápido](#)
 18. [Referencias](#)
-

1. Introducción: ¿Por qué Evaluar es Difícil?

El Problema de Fondo

Evaluar un modelo de clasificación de imágenes es relativamente simple: la imagen es un gato o no lo es. Evaluar un LLM es estructuralmente distinto:

- **No hay una sola "respuesta correcta"** en lenguaje natural. "Explícame la fotosíntesis" admite cientos de respuestas válidas, con distinto nivel de detalle, tono, longitud
- **Una salida puede ser fluida pero falsa** — la alucinación (vista en el Taller 1) es el enemigo silencioso: el texto suena bien pero es incorrecto
- **"Ser mejor" depende de la tarea y el caso de uso** — un modelo excelente para resumir contratos legales puede ser mediocre escribiendo poesía
- **Cientos de modelos nuevos cada año** → se necesita comparación objetiva y reproducible, no solo "lo probé y se sintió bien"

¿Qué Medimos Exactamente?

Dimensión	Pregunta que responde
Calidad / precisión	¿La respuesta es correcta?
Veracidad	¿Hay alucinaciones?
Seguridad y sesgo	¿El modelo discrimina o genera contenido dañino?
Latencia, costo, contexto	¿Es viable operacionalmente?

Este taller cubre las herramientas para responder cada una de estas preguntas con rigor, no con intuición.

Contaminación de Datos (Data Leakage / Benchmark Contamination)

Uno de los problemas más serios y menos discutidos en evaluación de LLMs:

Definición: ocurre cuando los ejemplos del benchmark (o muy similares) estuvieron en los datos de entrenamiento del modelo. El modelo "memoriza" en vez de razonar.

Consecuencias:

- Infla artificialmente los puntajes — el modelo no es "inteligente", solo "recuerda" la respuesta exacta que vio en entrenamiento
- Es la razón por la que benchmarks "viejos" se **saturan** (todos los modelos top superan 95%, perdiendo poder discriminativo)
- Esto crea una carrera permanente: cuando un benchmark se contamina/satura, la comunidad crea uno nuevo más difícil

Soluciones:

- Benchmarks **privados/holdout** (no publicados, para que no puedan estar en datos de entrenamiento)
- Benchmarks **"vivos"** que se actualizan constantemente (como LiveBench)
- Crear variantes de preguntas conocidas (cambiar números/nombres) para detectar memorización vs razonamiento real

Ejemplo de detección de contaminación:

Original: "¿Cuánto es 15% de 200?" → el modelo responde bien

Variante: "¿Cuánto es 17% de 340?" → si falla la variante pero
acierta la original → posible memorización, no razonamiento

2. Paradigmas de Evaluación

Tres Grandes Paradigmas

AUTOMÁTICA	HUMANA	LLM-AS-A-JUDGE
Métricas + benchmarks	Anotadores / preferencias	Un LLM evalúa a otro
Rápida, barata	Cara, "gold standard"	Escalable, pero con sesgos

Automática: aplicar fórmulas matemáticas (BLEU, exact match, etc.) o ejecutar el modelo contra un benchmark con respuestas conocidas. Es la más barata y rápida, pero la más limitada en captar matices de calidad.

Humana: personas reales juzgan o comparan respuestas. Es el "gold standard" porque captura matices que las métricas automáticas no pueden, pero es costosa, lenta y difícil de escalar.

LLM-as-a-judge: un modelo de lenguaje potente actúa como evaluador de las respuestas de otro modelo (o de sí mismo). Es el punto intermedio: mucho más barato y escalable que humanos, mucho más matizado que una fórmula, pero hereda los sesgos y limitaciones de los LLMs.

Otra Dimensión: ¿Cuándo se Evalúa?

Offline: con un dataset fijo, antes de desplegar el modelo a producción.

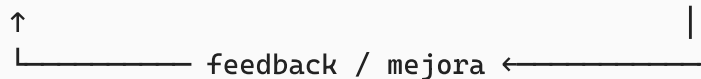
- Ejemplos: correr benchmarks académicos, tests automáticos en CI/CD
- Ventaja: reproducible, controlado, puedes comparar versiones exactamente

Online: con tráfico real en producción.

- Ejemplos: A/B testing entre dos versiones de un prompt, feedback de usuarios reales (pulgar arriba/abajo)
- Ventaja: refleja el uso real, captura casos que nunca imaginaste probar offline

Ciclo de vida completo:

Desarrollo → Evaluación Offline (CI/CD) → Deploy → Evaluación Online (producción)



3. Tipos de Evaluación: Capacidades vs. Tareas

Esta distinción es crítica y muchas veces se confunde en la práctica.

Evaluación por Capacidades

Mide habilidades **generales y transversales** del modelo: razonamiento, conocimiento, matemática, código, comprensión multilingüe.

Ejemplos: MMLU, GPQA, GSM8K.

Estas evaluaciones te dicen "qué tan bueno es el modelo en general", de forma comparable entre cualquier modelo del mundo.

Evaluación por Tareas

Mide el desempeño en una **tarea concreta de tu aplicación específica**.

Ejemplos: resumen de tickets de soporte, clasificación de emails, calidad de RAG sobre tus propios documentos legales.

Esto **requiere un dataset propio** — nadie más tiene exactamente tu caso de uso.

La Lección Más Importante de esta Sección

Un buen score en MMLU no garantiza buen desempeño en tu caso de uso.

Un modelo puede tener 90% en MMLU (conocimiento general excelente) y aun así fallar consistentemente resumiendo tus tickets de soporte específicos, porque esa tarea tiene su propio vocabulario, formato y matices que MMLU nunca midió.

Implicación práctica: los leaderboards públicos son útiles para elegir un modelo base candidato, pero **siempre** necesitas evaluar con tus propios datos antes de confiar en un modelo para producción.

4. Métricas Clásicas de NLP

4.1 Perplexity (Perplejidad)

Definición: mide qué tan "sorprendido" está el modelo ante un texto. Menor perplexity = mejor predicción del siguiente token.

$$L() = \exp \left(-\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(x_i | x_{i-1}) \right)$$

Intuición con ejemplo: dada la frase "El cielo es ____", el modelo debe predecir el último token.

Modelo	$p(\text{al})$	PPL = $1/p$	Interpretación
Modelo A (bueno)	0.8	1.25	Poco "sorprendido" → mejor
Modelo B (malo)	0.1	10	Muy "sorprendido" → peor

La intuición clave: la perplexity es el **número promedio de opciones entre las que el modelo "duda"**. PPL = 1 sería predicción perfecta; PPL = 10 equivale a elegir al azar entre ~10 palabras igualmente probables.

Ejemplo con una secuencia completa: "El cielo es azul"

Token	$p_{\theta}(x_i x_{i-1})$	$\log p$
El	0.40	-0.92
cielo	0.50	-0.69
es	0.70	-0.36
azul	0.80	-0.22
Media de log p		-0.55

$$L = \exp(0) \approx 1$$

Si las probabilidades fueran más bajas (modelo peor), la media de $\log p$ sería más negativa y la PPL subiría.

Limitación crítica: la perplexity solo mide qué tan bien el modelo predice el *siguiente token estadísticamente* — **no mide utilidad, veracidad ni seguimiento de instrucciones**. Un modelo puede tener excelente perplexity y aun así ser un asistente terrible (recuerda: pre-entrenamiento $\neq$ alineación, visto en el Taller 1).

4.2 Métricas Basadas en Referencia

Estas comparan la salida del modelo contra una respuesta "gold" (de referencia, considerada correcta).

Métrica	Tarea típica	Qué mide
BLEU	Traducción	Solapamiento de n-gramas (precisión)
ROUGE	Resumen	Solapamiento de n-gramas (recall)
METEOR	Traducción	N-gramas + sinónimos + stemming
Exact Match (EM)	QA	¿Coincide exactamente? (0/1)
F1 (token)	QA extractivo	Solapamiento de tokens
BERTScore	Generación	Similitud semántica (embeddings)
Pass@k	Código	¿Pasa los tests en k intentos?

Pass@k en detalle: se generan k soluciones distintas para el mismo problema de código; se cuenta como éxito si **al menos una** pasa los tests unitarios. Es la métrica base de HumanEval. Por ejemplo, Pass@10 mide si, dando 10 intentos al modelo, al menos uno funciona — relevante porque en la práctica un desarrollador puede pedir varias generaciones y elegir la mejor.

4.3 Limitaciones de las Métricas Clásicas

Esta es una de las secciones más importantes conceptualmente:

- **BLEU/ROUGE penalizan el parafraseo correcto:** si dices lo mismo con otras palabras, el solapamiento de n-gramas es bajo aunque el significado sea idéntico. Alta puntuación $\neq$ buena respuesta, y viceversa.
- **Exact Match es demasiado estricto:** "París" vs. "París, Francia" — ambas son correctas, pero EM las marca como distintas.

- **Requieren una referencia:** en tareas abiertas (un chatbot conversando) no existe "la" respuesta correcta única contra la cual comparar.
- **No capturan veracidad, coherencia ni seguridad:** una respuesta puede tener alto BLEU y seguir siendo falsa o peligrosa.

Por esto surgen los **benchmarks estandarizados** (sección 5-6) y el **LLM-as-a-judge** (sección 8) — para evaluar más allá del simple solapamiento textual.

Ejemplo ilustrativo del problema:

Referencia: "El gato está sobre la alfombra"
Candidata: "Sobre el tapete se encuentra el felino"

BLEU/ROUGE: MUY BAJO (casi ninguna palabra coincide)
Similitud semántica real: MUY ALTA (significan exactamente lo mismo)

Esto es exactamente el Ejercicio 1 que propone el curso: pedirle a un LLM que calcule el BLEU aproximado de este par y luego que juzgue la similitud semántica (0-1), para evidenciar la brecha entre ambos enfoques.

5. Benchmarks de Conocimiento y Razonamiento

Benchmark	Mide	Formato
MMLU	Conocimiento en 57 materias	Opción múltiple (4 opciones)
MMLU-Pro	Versión más difícil (10 opciones)	Opción múltiple
GPQA	Ciencia nivel posgrado "Google-proof"	Opción múltiple
ARC	Razonamiento científico escolar	Opción múltiple
HellaSwag	Sentido común / continuación de texto	Opción múltiple
BIG-Bench (Hard)	200+ tareas diversas	Mixto
GSM8K	Problemas matemáticos de primaria	Respuesta numérica
MATH	Matemática de competición	Respuesta numérica
Humanity's Last Exam (HLE)	Frontera de dificultad (nivel experto)	Mixto

Tendencia Clave: Saturación de Benchmarks

MMLU/HellaSwag están **saturados** (>90% en los mejores modelos). Hoy se usan GPQA, MMLU-Pro y HLE para diferenciar a los mejores modelos.

Esto es consecuencia directa del problema de contaminación de datos visto en la sección 1: cuando todos los modelos top superan el 90-95%, el benchmark ya no discrimina calidad — necesitas algo más difícil.

Anatomía de un Ítem de MMLU

Ejemplo (materia: biología universitaria):

¿Cuál de las siguientes moléculas almacena energía a corto plazo en la célula?
A) ADN B) ATP C) Colágeno D) Celulosa

- Se mide **accuracy**: % de respuestas correctas
- Suele evaluarse en **0-shot** o **few-shot** (con ejemplos en el prompt)
- El modelo elige comparando la **probabilidad** que asigna a cada letra de opción (A/B/C/D) — no necesariamente "razona" en texto libre

Anatomía de un Ítem de MMLU-Pro

Ejemplo (materia: física, 10 opciones):

Un bloque de 2 kg se desliza sin fricción por una rampa desde una altura de 5 m.
¿Cuál es su velocidad ($\approx$) al llegar a la base? ($g = 9.8 \text{ m/s}^2$)
A) 5.0 B) 7.0 C) 8.9 D) 9.9 E) 11.2
F) 14.0 G) 19.6 H) 22.0 I) 25.0 J) 49.0 (m/s)

Por qué MMLU-Pro es mejor que MMLU:

- **10 opciones** en vez de 4 → el acierto por azar baja de 25% a 10%, reduciendo el "ruido" de aciertos casuales
- Requiere **razonamiento real** (calcular $= \sqrt{2}$), no solo recordar un hecho memorizado
- Filtra preguntas triviales o ruidosas presentes en MMLU original → benchmark menos saturado

Anatomía de un Ítem de GPQA ("Google-proof Q&A")

Ejemplo (nivel posgrado, química cuántica):

En el desdoblamiento de campo cristalino de un complejo octaédrico d^6 de bajo espín, ¿cuál es la configuración electrónica de los orbitales t_{2g} y e_g , y el número de electrones desapareados?

- A) $t_{2g}^4 e_g^2$, 4 desapareados B) $t_{2g}^6 e_g^0$, 0 desapareados
C) $t_{2g}^5 e_g^1$, 2 desapareados D) $t_{2g}^3 e_g^3$, 6 desapareados

Lo que hace especial a GPQA:

- Escritas por **expertos con doctorado** — son difíciles incluso buscando en Google (de ahí el nombre)
- No-expertos aciertan ~34% (incluso con acceso a internet); expertos en el dominio aciertan ~65-74%
- Mide **conocimiento profundo y razonamiento**, no recuperación superficial de datos — esto es justo lo que un benchmark "a prueba de búsqueda" necesita capturar

Anatomía de un Ítem de Humanity's Last Exam (HLE)

Ejemplo (matemática/lógica, nivel experto):

Sea G un grupo finito simple no abeliano de orden menor a 1000. ¿Cuántos grupos de este tipo existen, y cuál es el de menor orden?

Respuesta esperada: 5 grupos; el menor es A_5 de orden 60.

Características de HLE:

- ~2,500 preguntas de **nivel experto** en muchas disciplinas (texto y multimodal)
- Diseñado deliberadamente para que **los mejores LLMs aún fallen la mayoría** — combate directamente el problema de saturación
- Formato de **respuesta exacta o opción múltiple verificable automáticamente** (no requiere un juez humano/LLM para calificar)
- Objetivo explícito: medir el **techo real de capacidad** en la frontera del conocimiento humano

6. Benchmarks de Código, QA y Seguridad

Benchmark	Dominio	Métrica
HumanEval	Generación de código	Pass@k
MBPP	Problemas Python básicos	Pass@k
SWE-bench	Resolver issues reales de GitHub	% resuelto
SQuAD	QA extractivo	EM / F1
TriviaQA	QA de conocimiento	EM / F1
DROP	QA con razonamiento discreto	F1
TruthfulQA	Veracidad / falsedades comunes	% veraz
ToxiGen	Detección de toxicidad	% tóxico
MMMU	Multimodal (texto + imagen)	Accuracy

SWE-bench: El Benchmark "Agéntico"

El modelo recibe un repositorio completo + un issue real de GitHub, y debe producir un **patch** (diff de código) que pase los tests existentes.

Esto conecta directamente con el Taller 2 (Agentes): SWE-bench no evalúa "¿el modelo sabe programar?" en abstracto, sino **¿puede el agente navegar un repositorio real, entender el contexto, identificar el bug, y producir una solución funcional?** — mide capacidades agénticas de ingeniería de software, no solo generación de snippets aislados.

TruthfulQA: Midiendo Falsedades Comunes

Este benchmark es particularmente interesante porque mide algo contraintuitivo: preguntas diseñadas para que la respuesta "popular" (la que la mayoría de humanos creería, por mitos comunes) sea **incorrecta**. Un modelo entrenado en texto de internet puede "aprender" a repetir mitos populares en vez de la verdad fáctica. TruthfulQA mide específicamente esta brecha.

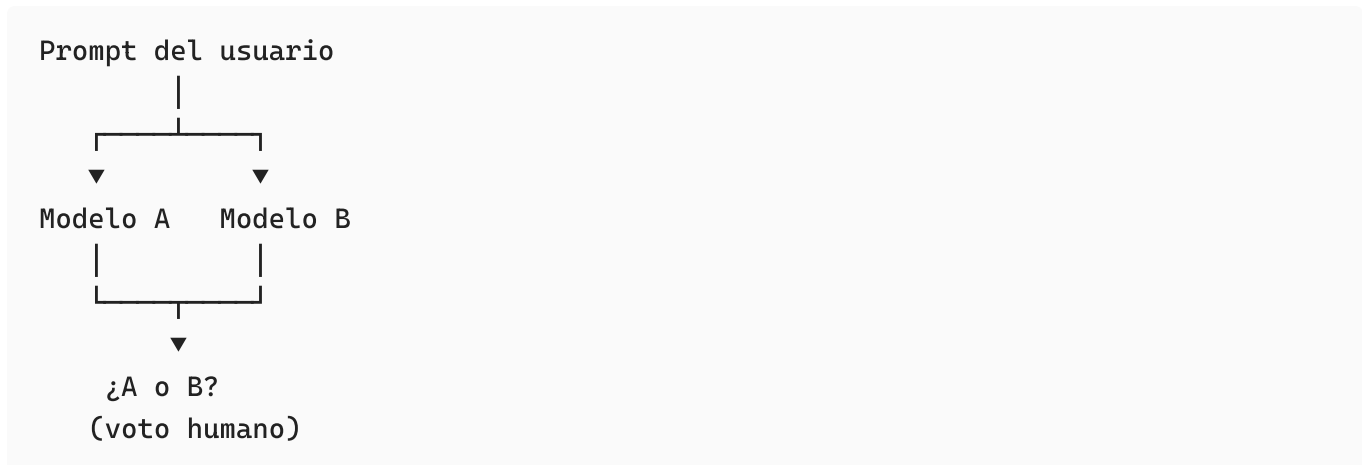
7. Evaluación Humana y Arenas

Chatbot Arena (LMArena)

¿Cómo funciona?

1. El usuario escribe un prompt
2. Recibe respuestas de **2 modelos anónimos** (A y B) — no sabe cuál modelo es cuál

3. Vota cuál respuesta es mejor
4. Se agregan **millones de votos** en un ranking **Elo**



Por qué este enfoque es valioso: a diferencia de un benchmark de opción múltiple, captura **preferencia humana real y holística** — incluyendo tono, formato, utilidad percibida — cosas que ningún benchmark automático mide directamente.

El Sistema Elo (del Ajedrez a los LLMs)

Idea: cada modelo tiene un puntaje. Tras cada "partida" (comparación), el ganador sube y el perdedor baja, según la **probabilidad esperada** de ganar.

Fórmulas:

$$E = \frac{1}{1 + 10^{(-)/00}}$$
$$\leftarrow + K(- E)$$

Donde:

- : puntaje Elo actual del modelo A
- : puntaje Elo actual del rival B
- E : probabilidad esperada de que A gane (entre 0 y 1)
- : resultado real de la "partida" (1 = A gana, 0 = A pierde, 0.5 = empate)
- K : factor de ajuste — cuánto cambia el puntaje por partida (típico $K = 2$)
- 00: constante de escala — una diferencia de 400 puntos implica ~10:1 de probabilidad de ganar

Cómo leer la fórmula (intuición clave):

- Si A es mucho mejor que B () $\rightarrow E \rightarrow 1$ (se espera casi con certeza que A gane)

- **Ganar a alguien peor sube poco el puntaje; ganar a alguien mejor sube mucho** (porque $-E$ es grande cuando ganas algo "inesperado")
- Si el resultado coincide exactamente con lo esperado, el puntaje casi no cambia — el sistema solo te "premia" por sorprender positivamente

Ejemplo numérico:

Modelo A: Elo = 1500

Modelo B: Elo = 1500 (mismo nivel inicial)

$E_A = 1 / (1 + 10^0) = 0.5$ (50% de probabilidad esperada)

Si A gana ($S_A = 1$):

$R_{A_nuevo} = 1500 + 32 \times (1 - 0.5) = 1500 + 16 = 1516$

Si A gana a un modelo mucho mejor ($R_B=1800$):

$E_A = 1/(1+10^{(300/400)}) \approx 0.18$ (solo 18% esperado de ganar)

$R_{A_nuevo} = 1500 + 32 \times (1 - 0.18) = 1500 + 26.2 = 1526.2$

→ subió MÁS porque la victoria fue "más sorprendente"

Ventaja: refleja preferencia humana real, agregada sobre millones de comparaciones.

Sesgos conocidos: verbosidad (respuestas más largas a veces "parecen" mejores aunque no lo sean), formato (markdown bonito influye en el voto), posición (a veces la posición A o B en pantalla influye en el voto).

8. LLM-as-a-Judge

La Idea Central

Se usa un modelo potente (ej. GPT-4, Claude Opus) como "juez" que puntúa o compara respuestas según una **rúbrica** especificada en el prompt.

Implementaciones conocidas:

- **MT-Bench:** 80 preguntas multi-turno, el juez puntúa de 1 a 10
- **AlpacaEval:** calcula el "win-rate" de un modelo vs. un modelo de referencia fijo

Por qué es tan usado: es **escalable y barato** comparado con anotación humana — puedes evaluar miles de respuestas en minutos en vez de contratar anotadores durante semanas.

Sesgos Conocidos del LLM-as-a-Judge

Sesgo	Descripción
Posición	El juez tiende a favorecer la primera respuesta que ve (A), independiente de su calidad real
Verbosidad	Respuestas más largas tienden a recibir mejor puntaje, incluso si no son mejores
Auto-preferencia	Un modelo evaluando respuestas (incluso anónimas) tiende a preferir el estilo de respuestas similar al suyo propio

Mitigación: intercambiar el orden de las respuestas (evaluar A vs B, y luego B vs A) y usar rúbricas explícitas y claras en el prompt del juez.

Plantilla Típica de un Prompt de Juez

Eres un evaluador experto e imparcial. Compara las dos respuestas a la PREGUNTA según: (1) correctitud factual, (2) utilidad, (3) claridad.

PREGUNTA: {pregunta}
 RESPUESTA A: {respuesta_a}
 RESPUESTA B: {respuesta_b}

Razona paso a paso brevemente y termina con una única línea:
 VEREDICTO: A | B | EMPATE

Punto crítico de la metodología: para reducir el sesgo de posición, se ejecuta el juicio **dos veces, intercambiando A y B**. Si el veredicto cambia según el orden, eso es evidencia directa de sesgo de posición contaminando la evaluación — y deberías desconfiar del resultado o promediar ambas corridas.

Esto es exactamente el Ejercicio 4 del taller: generar la misma respuesta con 2 modelos, usar el prompt de juez en un tercer modelo, ejecutar 2 veces intercambiando el orden, y verificar si el veredicto es estable.

9. Métricas para RAG

¿Por Qué No Bastan los Benchmarks Académicos?

En una aplicación real basada en RAG (Retrieval-Augmented Generation, visto en el Taller 1):

- **No hay opción múltiple** — las respuestas son abiertas

- **Importa si la respuesta se apoya en el contexto recuperado** (no inventa información que no estaba en los documentos)
- **Importa la relevancia** de lo que se recuperó y la **ausencia de alucinaciones**

⇒ Se necesitan métricas específicas, muchas **sin referencia** (reference-free — no necesitan una "respuesta gold" para comparar), evaluadas frecuentemente por LLM-as-a-judge.

Métricas Clave para RAG

Métrica	Qué responde
Faithfulness / Groundedness	¿La respuesta se apoya en el contexto recuperado (no alucina)?
Answer Relevancy	¿La respuesta es pertinente a la pregunta hecha?
Context Precision	De lo recuperado, ¿cuánto es realmente relevante?
Context Recall	¿Se recuperó toda la información necesaria para responder bien?
Answer Correctness	¿Coincide con la respuesta esperada (gold)?
Hallucination	¿Contiene afirmaciones no soportadas por el contexto?

Pipeline RAG y dónde se mide cada métrica:

```

Pregunta → Retriever → Contexto → LLM → Respuesta
           |                   |
           |                   |
context precision/recall    faithfulness
(¿lo recuperado es bueno?) (¿usó bien el contexto?)
  
```

Diagnóstico clave que permiten estas métricas: si tu RAG da malas respuestas, puedes diagnosticar **dónde** está el problema:

- ¿Context Precision/Recall bajo? → el problema está en el **retriever** (no encuentra los documentos correctos)
- ¿Faithfulness baja pero Context Recall alto? → el problema está en el **LLM generador** (tenía el contexto correcto pero igual alucinó o lo ignoró)

Esta separación de responsabilidades es una de las contribuciones más útiles de las métricas RAG modernas — sin ellas, "el RAG da malas respuestas" es un diagnóstico inútil para saber qué arreglar.

G-Eval: Métricas a Medida con LLM

Idea: defines un criterio en lenguaje natural y el LLM genera pasos de evaluación (chain-of-thought) y asigna un puntaje. Permite crear **métricas personalizadas** para cualquier dimensión que te importe.

Ejemplo de criterio custom:

"Evalúa la cortesía de la respuesta del agente de soporte en una escala de 1 a 5, penalizando respuestas cortantes o que culpen al cliente."

G-Eval es la base conceptual de muchas métricas custom en frameworks como DeepEval y LangSmith — cuando una métrica estándar (faithfulness, relevancy) no captura lo que te importa específicamente (ej. "tono de marca", "empatía", "cumplimiento de un guion de ventas"), G-Eval te permite definir tu propio criterio y delegarlo a un LLM-juez con razonamiento explícito.

10. Métricas para Agentes

¿Por Qué Evaluar Agentes es Distinto?

Un agente no solo responde: **actúa**. Razona, decide qué herramientas usar, en qué orden, con qué argumentos, y combina los resultados en varios pasos.

Esto conecta directamente con el Taller 2: evaluar un agente no es lo mismo que evaluar un LLM simple porque:

- **No basta evaluar la respuesta final** — hay que evaluar el **proceso** (la *trayectoria*)
- Una respuesta correcta puede venir de un **camino ineficiente** o incluso **por suerte** (el agente llamó 5 tools innecesarias y al final acertó)
- Una respuesta incorrecta puede deberse a una **tool mal elegida** o **mal parametrizada**, no a un fallo de razonamiento general

⇒ Se evalúan dos planos: **resultado** (outcome) y **trayectoria** (proceso).

Métricas Clave para Agentes

Métrica	Qué responde
Tool Correctness	¿Llamó a las herramientas correctas (vs. las esperadas)?
Tool Selection	¿Eligió la tool adecuada para la subtask específica?
Parameter Accuracy	¿Pasó los argumentos correctos a la tool?

Métrica	Qué responde
Trajectory / Order	¿Ejecutó los pasos en el orden correcto?
Task Completion	¿Logró el objetivo final del usuario?
Step / Reasoning	¿Cada paso intermedio fue válido y necesario?
Efficiency	Número de pasos, tokens y costo, vs. lo óptimo

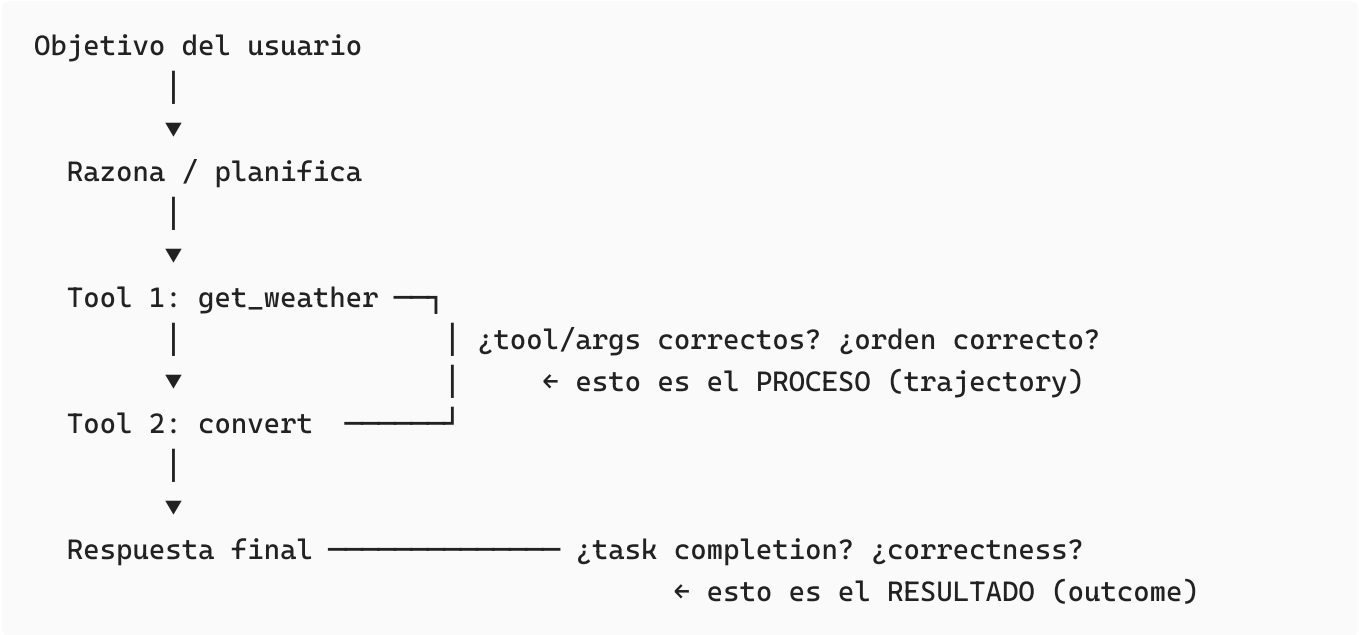
Ejemplo guía: "¿Qué clima hace en Lima y convierte 20°C a °F?"

Tools esperadas (en orden): `get_weather("Lima")` → `celsius_to_fahrenheit(20)`

Se verifica:

- ¿Llamó ambas tools? → Tool Correctness
- ¿Con los argumentos correctos? → Parameter Accuracy
- ¿En ese orden? → Trajectory
- ¿Completó la tarea final? → Task Completion

Outcome vs. Proceso (Trajectory)



Por qué importan ambos planos por separado: imagina un agente que, para responder "¿qué clima hace en Lima?", llama primero a una tool irrelevante de calendario, luego corrige y llama a `get_weather`, y finalmente da la respuesta correcta. El **outcome** es perfecto (respuesta correcta), pero la **trayectoria** reveló ineficiencia — gastó tokens y tiempo de más, y en un sistema con acciones costosas (ej. transferencias bancarias) ese paso de más podría haber sido peligroso.

Ejemplo Práctico con DeepEval: Tool Correctness


```
from deepeval.test_case import LLMTestCase, ToolCall
from deepeval.metrics import ToolCorrectnessMetric

caso = LLMTestCase(
    input="Clima en Lima y convierte 20C a F",
    actual_output="En Lima hay 20C, equivalentes a 68F.",
    # herramientas que el agente realmente llamó
    tools_called=[ToolCall(name="get_weather"),
ToolCall(name="convert_temp")],
    # herramientas que esperábamos que llamara
    expected_tools=[ToolCall(name="get_weather"),
ToolCall(name="convert_temp")],
)

metric = ToolCorrectnessMetric() # compara tools_called vs expected_tools
metric.measure(caso)
print(metric.score, metric.reason) # 1.0 -> usó las tools correctas
```

Este patrón de testing — definir explícitamente qué tools/argumentos/orden esperas, y comparar contra lo que el agente realmente hizo — es la base de cualquier suite de evaluación seria para agentes en producción.

11. Frameworks de Evaluación

Panorama General

Framework	Enfoque	Cuándo usar
DeepEval	Estilo pytest; 14+ métricas (G-Eval, RAG, alucinación)	Tests unitarios de LLM / CI
RAGAS	Específico para pipelines RAG	Evaluar retrieval + generación
Im-eval-harness	Benchmarks académicos (MMLU, etc.)	Comparar modelos base
OpenAI Evals	Framework de evals + registry	Evals personalizadas
HELM (Stanford)	Evaluación holística multi-métrica	Reportes amplios
promptfoo	Test de prompts (YAML), side-by-side	Iterar prompts rápido
Langfuse / TruLens	Observabilidad + evals en producción	Monitoreo online

Regla Práctica de Decisión

```
Benchmarks de modelo base → lm-eval-harness
Calidad de tu aplicación   → DeepEval / RAGAS
Producción (monitoreo)     → LangSmith / Langfuse
```

Esta regla resume todo el taller en una línea: hay herramientas distintas para preguntas distintas. No uses lm-eval-harness para evaluar tu app de soporte al cliente (no fue diseñado para eso), y no uses MMLU para decidir si tu RAG funciona bien.

DeepEval: "Pytest para LLMs"

Conceptos centrales:

- `LLMTestCase` : estructura que contiene `input` , `actual_output` , `expected_output` , `context`
- **Métricas**: `AnswerRelevancyMetric` , `FaithfulnessMetric` , `HallucinationMetric` , `GEval` , entre otras 14+
- `assert_test` : falla el test si la métrica cae por debajo de un umbral (`threshold`)
- **Integración con pytest y CI/CD**: se ejecuta como cualquier test de software

Ventaja central: conviertes la evaluación de calidad de LLM en **tests automáticos** — cada cambio de prompt o de modelo se re-evalúa solo, sin intervención manual, igual que cualquier suite de tests de software tradicional.

Ejemplo de código:

```
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import AnswerRelevancyMetric, FaithfulnessMetric

def test_rag():
    caso = LLMTestCase(
        input="¿Cuál es la capital de Perú?",
        actual_output="La capital de Perú es Lima.",
        retrieval_context=["Lima es la capital y ciudad más grande de Perú."]
    )
    assert_test(caso, [
        AnswerRelevancyMetric(threshold=0.7),
        FaithfulnessMetric(threshold=0.7),
    ])
```

Se ejecuta con: `deepeval test run test_rag.py`

RAGAS: Evaluación Especializada de RAG

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision

dataset = {
    "question": ["¿Qué es un LLM?"],
    "answer": ["Un modelo de lenguaje grande..."],
    "contexts": [["Un LLM es una red neuronal entrenada con texto..."]],
    "ground_truth": ["Un modelo de lenguaje grande (LLM)..."]
}

resultado = evaluate(dataset,
    metrics=[faithfulness, answer_relevancy, context_precision])

print(resultado) # {'faithfulness': 0.95, 'answer_relevancy': 0.88, ...}
```

Valor diferencial de RAGAS: es ideal para diagnosticar **si el problema está en el retriever o en el generador** dentro de un pipeline RAG, gracias a la separación de métricas (context precision/recall vs faithfulness) explicada en la sección 9.

12. LangSmith para Evaluación

LangSmith ya se cubrió en el Taller 2 desde la perspectiva de **observabilidad de agentes**. Aquí se profundiza en su rol específico para **evaluación sistemática**.

Los Cuatro Pilares de LangSmith para Evaluación

1. **Tracing:** registra cada paso (prompts, tool calls, tokens, latencia) — la base de cualquier observabilidad
2. **Datasets:** conjuntos de ejemplos (input + reference) reutilizables para evaluar versiones distintas de tu app
3. **Evaluators:** funciones (o LLMs-juez) que puntúan cada salida de tu aplicación contra el dataset
4. **Experiments:** corres tu aplicación completa sobre un dataset y comparas resultados entre versiones (ej. prompt v1 vs prompt v2)

```
Dataset → App/Chain → Evaluators → Dashboard (comparar)
(ejemplos) (tu app) (puntúan) (analizas resultados)
```

Métricas Observables en LangSmith

De calidad:

- Correctness (vs. referencia)
- Relevance, helpfulness
- Faithfulness / groundedness
- Métricas custom vía LLM-as-judge

Operacionales:

- Latencia (P50, P99)
- Tokens y costo (\$)
- Tasa de error
- **Trajectory de agentes** (pasos/tools — conecta directamente con la sección 10)
- Feedback humano (positivo/negativo)

Ejemplo de Código: Evaluación con LangSmith

```
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()

# 1. Crear dataset
dataset = client.create_dataset("preguntas_capitales")
client.create_examples(
    inputs=[{"q": "¿Capital de Perú?"}],
    outputs=[{"a": "Lima"}],
    dataset_id=dataset.id)

# 2. Evaluator LLM-as-judge (correctness)
def correctness(run, example):
    score = run.outputs["a"].strip() == example.outputs["a"]
    return {"key": "correctness", "score": int(score)}

# 3. Correr experimento
evaluate(lambda x: mi_app(x["q"]),
    data="preguntas_capitales",
    evaluators=[correctness])
```

Este patrón — dataset fijo + evaluator + comparación de experimentos — es exactamente lo que necesitas para responder con datos (no intuición) preguntas como "¿el prompt nuevo es

mejor que el viejo?" o "¿vale la pena cambiar de modelo?".

13. Leaderboards Públicos

¿Dónde Comparar Modelos?

Leaderboard	Qué ofrece
LMarena	Ranking Elo por preferencia humana (sección 7)
Artificial Analysis	Índices de inteligencia, velocidad, precio; "Agentic Index"
Vellum LLM Leaderboard	Comparador lado a lado (MMLU, GPQA, costo, contexto)
LiveBench	Benchmark "vivo", se actualiza para evitar contaminación
Open LLM Leaderboard (HuggingFace)	Modelos open-weights con suite estándar

Artificial Analysis: El "Agentic Index"

Artificial Analysis combina varios benchmarks en **índices** y añade ejes operacionales (velocidad y precio).

- **Intelligence Index:** agregado de razonamiento, conocimiento, matemática, código — una sola cifra resumen de "qué tan inteligente" es el modelo en general
- **Agentic Index:** enfocado específicamente en tareas **agénticas** (uso de herramientas, multi-paso) usando benchmarks como SWE-bench, Terminal-Bench, t-bench
- Permite **filtrar open-weights vs. proprietary**
- Ejes adicionales: **output speed** (tokens/segundo) y **precio** (\$/M tokens)

Por qué importa el Agentic Index específicamente: un modelo puede tener excelente Intelligence Index (gran conocimiento general) pero mediocre Agentic Index (no sabe usar herramientas eficientemente) — si estás construyendo un agente (Taller 2), el Agentic Index es mucho más predictivo de tu éxito que el ranking general.

Comparativa Ilustrativa de Modelos (junio 2026)

Modelo	MMLU-Pro	GPQA	Contexto	Tipo
Claude Opus 4.8	90%	86%	1M	Proprietary
GPT-5.5	89%	85%	1.05M	Proprietary
Gemini 3.5 Pro	88%	84%	2M	Proprietary
Claude Sonnet 4.6	86%	80%	1M	Proprietary
DeepSeek V4	85%	79%	1M	Open-weights
Qwen 3.5 (235B)	83%	76%	256K	Open-weights
Llama 4 Maverick	82%	75%	1M	Open-weights

Valores ilustrativos con fines didácticos — consultar los leaderboards en vivo para cifras reales y actualizadas.

Lección de esta tabla: los modelos open-weights ya están a una distancia muy pequeña de los propietarios top en estas métricas — la decisión de cuál usar depende cada vez más de costo, latencia, y control sobre los datos (self-hosting) que de una brecha de "inteligencia" significativa.

14. Dimensiones Más Allá de la Precisión

Dimensiones Operacionales

Dimensión	Qué mide
Latencia	TTFT (time to first token), tokens/segundo (visto en Taller 1)
Throughput	Peticiones concurrentes que el sistema soporta
Costo	\$/M tokens, separado por entrada y salida
Contexto máximo	Cuánto puede procesar de una vez (Taller 1)

Dimensiones de Confianza

Dimensión	Qué mide
Alucinaciones y veracidad	¿Inventa información? (Taller 1, sección de limitaciones)
Robustez	¿Resiste prompts adversariales sin romperse?
Sesgo y toxicidad	¿Genera contenido discriminatorio o dañino?

Dimensión	Qué mide
Seguridad (jailbreaks)	¿Puede ser manipulado para saltarse sus restricciones?

La Conclusión Central de esta Sección

El "mejor" modelo es el que optimiza el trade-off para tu caso de uso.

No existe "el mejor LLM" en abstracto. Existe el mejor LLM **para tu aplicación específica, con tu presupuesto, tu tolerancia a latencia, y tus requisitos de seguridad**. Un modelo de máxima inteligencia pero altísimo costo es la elección correcta para un asistente legal de alto valor, y la elección incorrecta para un chatbot de FAQ de alto volumen.

15. Buenas Prácticas y Resumen

Checklist de Buenas Prácticas (síntesis del taller)

1. **Elige el benchmark/métrica según el caso de uso, no la moda** — no uses GPQA para decidir si tu chatbot de soporte es bueno
2. **Construye un dataset propio** con ejemplos reales de tu aplicación — los leaderboards públicos no conocen tu dominio
3. **Combina offline (CI con DeepEval) + online (LangSmith en producción) + evaluación humana** — ningún método solo es suficiente
4. **Cuidado con el overfitting a benchmarks** y la contaminación de datos (sección 1)
5. **Controla los sesgos del LLM-as-a-judge** (posición, verbosidad) — intercambia el orden, usa rúbricas claras
6. **Reporta también costo y latencia, no solo accuracy** — un sistema "preciso" pero inviable económicamente no sirve

Tabla-Resumen de Decisión

Necesito...	Uso...
Comparar modelos base	MMLU/GPQA, lm-eval-harness, leaderboards
Evaluar mi RAG	RAGAS, DeepEval (faithfulness, relevancy)
Tests en CI/CD	DeepEval (assert_test)
Monitorear producción	LangSmith / Langfuse
Preferencia humana	LMarena (Elo), evaluación humana directa
Decidir costo/velocidad	Artificial Analysis, Vellum

Esta tabla es, en esencia, el mapa mental completo del taller — cada fila resume una sección entera del curso en una decisión práctica.

16. Ejercicios del Taller

Ejercicio 0: Calentamiento

Pregunta a **ChatGPT**, **Gemini** y **Claude**: *"Lista 3 limitaciones de los benchmarks de LLMs y explica brevemente cada una"*. Compara las respuestas — ¿coinciden? ¿cuál es más concreta?

Ejercicio 1: BLEU vs. Semántica

Usa el par de frases con mismo significado, distintas palabras (sección 4.3). Pide a un LLM que calcule el BLEU aproximado y luego que juzgue la similitud semántica (0-1). Discute la diferencia — esto evidencia directamente la limitación de las métricas clásicas.

Ejercicio 2: Reto de Contaminación

Toma una pregunta tipo MMLU, crea una **variante** (cambia números/nombres manteniendo la lógica). Pregunta ambas versiones a un LLM. Si falla la variante pero acierta la original → evidencia de memorización en vez de razonamiento real.

Ejercicio 4: Sé el Juez

Genera la misma respuesta con 2 modelos distintos. Usa el prompt de juez (sección 8) en un tercer modelo. Ejecuta 2 veces intercambiando el orden A/B. Si el veredicto cambia → evidencia de sesgo de posición.

Ejercicio 5: DeepEval

En Colab: `pip install deepeval`. Crea 3 `LLMTestCase` (uno correcto, uno con alucinación, uno irrelevante). Corre `AnswerRelevancy` y `Faithfulness`. ¿Las métricas detectan el caso con alucinación? Revisa los "reasons" que da cada métrica.

Ejercicio 6: LangSmith

Crea cuenta y API key en `smith.langchain.com`. (1) Crea un dataset con 5 preguntas. (2) Define un evaluador de correctness (LLM-as-judge). (3) Corre 2 prompts distintos y compara experimentos en el dashboard. ¿Qué prompt gana en correctness? ¿Cuál es más barato/rápido?

Ejercicio 7: Elegir un Modelo

En **Vellum** y **Artificial Analysis**, elige el mejor modelo para 3 escenarios:

- 1. Chatbot de soporte de alto volumen (prioriza costo/velocidad)
- 2. Asistente de investigación científica (prioriza GPQA/razonamiento)
- 3. Agente de código (prioriza Agentic Index/SWE-bench)

Justifica cada elección con métricas concretas del leaderboard.

Ejercicio: Evaluar un Agente

Crea un agente simple con 2 tools (ej. buscar y calcular). Haz una pregunta que requiera **ambas** en cierto orden. Evalúa con Tool Correctness y Task Completion. Luego dale una pregunta que solo necesite **una** tool: ¿llama de más (ineficiencia)? Mide la eficiencia (pasos/tokens).

17. Glosario Rápido

Término	Significado
Perplexity (PPL)	Qué tan "sorprendido" está el modelo prediciendo un texto; menor = mejor
BLEU/ROUGE	Métricas de solapamiento de n-gramas contra una referencia
Pass@k	Éxito si al menos 1 de k generaciones pasa los tests (código)
MMLU / GPQA / HLE	Benchmarks de conocimiento/razonamiento, en orden creciente de dificultad
Contaminación de datos	Cuando ejemplos del benchmark estaban en el entrenamiento — infla puntajes
Elo	Sistema de puntaje relativo basado en comparaciones pareadas (de ajedrez)
LLM-as-a-judge	Un LLM evalúa/puntúa las respuestas de otro modelo
Faithfulness / Groundedness	¿La respuesta RAG se apoya en el contexto recuperado?
Trajectory (de agentes)	La secuencia de pasos/tools que siguió un agente, evaluada como proceso
G-Eval	Framework para crear métricas custom vía LLM con criterio en lenguaje natural
Reference-free	Métrica que no necesita una respuesta "gold" para evaluar

18. Referencias

Papers Citados en el Curso

1. Papineni et al. — "BLEU: a Method for Automatic Evaluation of Machine Translation" — ACL, 2002
2. Zhang et al. — "BERTScore: Evaluating Text Generation with BERT" — ICLR, 2020
3. Hendrycks et al. — "Measuring Massive Multitask Language Understanding" (MMLU) — ICLR, 2021
4. Rein et al. — "GPQA: A Graduate-Level Google-Proof Q&A Benchmark" — COLM, 2024
5. Cobbe et al. — "Training Verifiers to Solve Math Word Problems" (GSM8K) — arXiv, 2021
6. Chen et al. — "Evaluating Large Language Models Trained on Code" (HumanEval) — arXiv, 2021
7. Jimenez et al. — "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" — ICLR, 2024
8. Lin et al. — "TruthfulQA: Measuring How Models Mimic Human Falsehoods" — ACL, 2022
9. Chiang et al. — "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference" — ICML, 2024
10. Zheng et al. — "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" — NeurIPS, 2023
11. Liu et al. — "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment" — arXiv, 2023
12. Es et al. — "RAGAS: Automated Evaluation of Retrieval Augmented Generation" — arXiv, 2023

Herramientas y Plataformas

- [DeepEval](#) — framework "pytest para LLMs"
- [RAGAS](#) — evaluación especializada de RAG
- [LangSmith](#) — observabilidad + evaluación
- [LMArena](#) — ranking Elo por preferencia humana
- [Artificial Analysis](#) — índices de inteligencia/velocidad/precio
- [Vellum LLM Leaderboard](#)
- [LiveBench](#)
- [Open LLM Leaderboard \(HuggingFace\)](#)

Este documento integra los contenidos del Taller 5 con expansiones conceptuales, ejemplos numéricos y de código para construir una comprensión sólida y aplicable de la evaluación de LLMs y sus aplicaciones (RAG, agentes).